dlaczego robimy zmianę

refactor-opportunities

Notatka kontekstowa dla laika — od mapy projektu do decyzji „co naprawiamy”

Repozytorium: apache/maven (Apache Maven 4)
Zmiana: context/changes/refactor-opportunities
Status: plan przejrzany (plan_reviewed) · 2026-10-14
Źródło prawdy: repo-map → model-builder-flow-analysis → research → plan K2

W jednym zdaniu

Nie przebudowujemy całego Mavena. Najpierw zmapowaliśmy, gdzie system naprawdę boli. Potem znaleźliśmy konkretny dług w „budowaniu modelu POM”. Ta zmiana odpowiada na pytanie: które z tych problemów warto zabezpieczyć najpierw — i zaczyna od najtańszego, ale krytycznego zabezpieczenia: żeby nikt przez pomyłkę „nie naprawił” celowej różnicy między Mavenem 3 a Mavenem 4.

Spis treści (ścieżka rozumowania)

1. Czym jest Maven — bez żargonu
2. Skąd się wzięła mapa projektu (repo-map)
3. Co mapa mówi o ryzyku
4. Poprzedni krok: analiza przepływu ModelBuildera
5. Czym jest zmiana refactor-opportunities
6. Trzej kandydaci (K1, K2, K3) — w prostych słowach
7. Dlaczego wybraliśmy właśnie K2
8. Co konkretnie zrobimy (i czego świadomie nie robimy)
9. Jak to się ma do całego M4 / „realizing change”
10. Podsumowanie dla kogoś spoza zespołu

Ta notatka idzie od szerokiego obrazu do wąskiej decyzji. Jeśli czytasz tylko jedną stronę — zacznij od ramki na okładce i sekcji 7–8.

1. Czym jest Maven — bez żargonu

Wyobraź sobie, że budujesz dom z tysiąca elementów od różnych dostawców. Ktoś musi przeczytać instrukcję („jakie ściany, jakie okna, w jakiej kolejności"), ściągnąć brakujące części i złożyć wszystko w działający budynek. Apache Maven robi coś podobnego dla projektów Java: czyta plik konfiguracyjny zwany POM (Project Object Model — opis projektu), rozwiązuje zależności i uruchamia kroki budowania (kompilacja, testy, pakowanie).

Wersja, nad którą pracuje społeczność w tym repozytorium, to przede wszystkim Maven 4 — duża modernizacja względem Mavena 3. Stary i nowy świat muszą jeszcze przez jakiś czas współistnieć: stare pluginy, stare zachowania i nowe, ostrzejsze reguły. To napięcie (most M3↔M4) jest kluczowe dla zrozumienia naszej zmiany.

2. Skąd się wzięła mapa projektu (repo-map)

Zanim ktokolwiek zaczął „refaktorować", powstała mapa onboardingowa: context/map/repo-map.md. Jej cel nie brzmi jak „naprawmy kod", tylko: gdzie żyje system, co boli, od czego czytać, kogo zapytać.

Mapa powstała z syntezy trzech artefaktów (historia gita, struktura zależności, mapa ludzi) w oknie ~12 miesięcy. Ważne ograniczenie: to nie jest pełne przeczytanie każdej linii kodu — to radar ryzyka i aktywności, żeby nie marnować energii na fałszywe centra uwagi (np. gęste poprawki w testach integracyjnych albo Dependabot).

Jak Maven jest ułożony (warstwy)

Mapa pokazuje przewidywalny układ warstw — jak piętra budynku, gdzie niższe nie powinny zależeć od wyższych:

Warstwa	Rola w prostych słowach
cli / embedder	Cienkie wejście — tu użytkownik odpala mvn
core	Runtime: kolejkowanie celów, projekty, pluginy
impl	Budowanie modelu POM + implementacje usług
api/*	Publiczny kontrakt Mavena 4 (to, czego trzyma się ekosystem)
compat/* + support	Most do Mavena 3 i wspólny I/O modelu
its/	Lustro regresji — dużo ruchu, ale nie „serce produktu"

Praca (po odfiltrowaniu botów) skupia się w kolejności: impl → core → ITS → cli. Boli przede wszystkim: edge case'y modelu (profile, parent, CI-friendly wersje), splątany lifecycle w core oraz każdy ruch w publicznym API.

3. Co mapa mówi o ryzyku

Mapa wyróżnia pięć stref ryzyka. Dla naszej historii najważniejsze są dwie pierwsze i most kompatybilności:

Strefa 1 — Budowanie modelu POM

To #1 aktywność i wysoki odsetek poprawek. Kod żyje głównie w impl/maven-impl (hub: DefaultModelBuilder), ze stykiem do legacy compat/maven-model-builder (~88% zmian w starym builderze idzie razem z impl). Tu rozstrzygane są profile, parent POM-y i przyjazne CI wersje w stylu ${revision}.

Strefa 2 — Lifecycle, reactor, pluginy

W impl/maven-core siedzi orkiestracja budowania wielu projektów naraz. „Mała” lokalna zmiana często tyka kilku pakietów naraz (cykle wewnętrzne).

Strefa 4 — Most M3↔M4

Świadomie utrzymywane mosty w compat/* i wspólnym I/O. Jedna zmiana w czytaniu/pisaniu modelu uderza w nowy i stary świat naraz. To nie jest „brud do wycięcia jutro” — to load-bearing kompromis podczas migracji ekosystemu.

> Wniosek z mapy (dla laika)
>
> Jeśli chcesz bezpiecznie poprawiać Maven 4, nie zaczynaj od najgłośniejszych plików w gicie. Zaczynaj od ścieżki: jak z pliku POM powstaje „model projektu”, i od tego, jak nowy kod dogaduje się ze starym mostem kompatybilności.

4. Poprzedni krok: analiza przepływu ModelBuildera

Zmiana model-builder-flow-analysis była Deep Focusem na strefę 1 z mapy. Jej intencja: prześledzić przepływ budowania modelu POM — od odczytu pliku do przejścia model → project — zaczynając od hubu DefaultModelBuilder.

Ta analiza celowo nie zamykała pytania „co naprawiamy". Udokumentowała dług techniczny i ryzyka strukturalne: ogromny hub, bliźniaczy kod M3/M4, luki w testach, kruche sprzężenia. Zostawiła otwarte: które z tych problemów warto naprawić, w jakim kształcie i w jakiej kolejności.

Bez tego kroku „refactor-opportunities" byłoby strzelaniem w ciemno. Z nim mamy listę problemów z dowodami w kodzie i historii commitów.

5. Czym jest zmiana refactor-opportunities

To kolejny etap tej samej historii — nie nowy produkt, tylko uporządkowanie długu w okazje refaktoru i decyzja, którą okazję realizujemy.

Etapy (jak w change.md)

- Eksploracja — badamy każdy zapisany problem; zero refaktoru, zero decyzji produktowej; wynik: research.md z rankingiem.
- Decyzja i plan — wybieramy opportunity i piszemy plan implementacji (plan.md + brief).
- Implementacja — dopiero według przyjętego planu (jeszcze nie startuje w tej notatce).

Status dziś: plan_reviewed — research kompletny, plan przejrzany jako SOUND, gotowy do realizacji faz K2.

6. Trzej kandydaci (K1, K2, K3) — w prostych słowach

Research wyłonił trzech kandydatów strukturalnych. Reszta problemów (Jacoco %, luki mapy, unknown SPI) to koszt / osłony, nie „refaktor do zrobienia teraz".

K1 — „Kuchnia, w której wszystko stoi na jednym stole"

Klasa DefaultModelBuilder w impl ma ~3450 linii i ~19 współpracowników w konstruktorze. To świadomy orchestrator pipeline'u (tak miał być), ale rozmiar pliku urósł akumulacyjnie. Docelowo: wydzielić etapy do mniejszych typów w tym samym pakiecie (bez nowego publicznego API). Wartościowe, ale droższe i ryzykowne bez dobrych testów charakterystyki.

K2 — „Bliźnięta, które mają się różnić — i ktoś może to „naprawić""

Istnieją dwa DefaultModelBuilder: nowy (impl / M4) i stary (compat / M3, oznaczony jako deprecated). Nie są spięte kompilatorem — semantyka trzeba uzgadniać ręcznie. Przy konkretnym błędzie (niespójny parent relativePath vs groupId/artifactId) M4 domyślnie krzyczy FATAL, a compat zostaje przy WARNING. Dodatkowo flaga maven.maven3Personality celowo łagodzi zachowanie M4 do stylu M3. To nie bug — to kontrakt migracyjny. Brakuje jednak testów, które chronią przed fałszywym „fix twin" (wyrównaniem severity „bo wygląda niespójnie").

K3 — „Piętro nad kuchnią: składanie projektu w reactorze"

DefaultProjectBuilder w core (~1420 LOC) materializuje MavenProject na modelu. Warstwa jest świadoma od 2009 — nie należy wciągać jej z powrotem do ModelBuildera. Dług jest w gęstości orkiestracji i cienkich testach, nie w samym istnieniu warstwy. Niższy priorytet, dopóki nie ma characterization ścieżki reactora.

Ranking z researchu (propozycja, nie ostateczna decyzja produktu)

#	Opportunity	Dlaczego w tej kolejności
1	K2 — guardy twin / personality	Najniższy koszt zmiany, wysoki koszt błędu (fałszywy fix kontraktu)
2	K1 — extract stage'ów hubu	Największy lokalny dług; najpierw guardy z K2
3	K3 — extract BuildSession	Świadoma warstwa; najpierw characterization

7. Dlaczego wybraliśmy właśnie K2

Plan (po wywiadzie / decyzji) bierze wyłącznie pełne K2 — nie K1 i nie K3 w tej zmianie. Powody, w języku codziennym:

1) Najpierw zabezpieczamy sens, potem ruszamy meble

Zanim ktoś zacznie rozbijać 3450-liniowy hub (K1), musi być jasne, które zachowania są celowe. Bez guardów refaktor może „wyrównać" FATAL i WARNING i zepsuć kontrakt M4/M3 — wyglądając jak poprawka spójności.

2) Najtańszy blast radius

Ścieżka guard: dokumentacja + testy (+ ewentualnie 1–3 linie komentarza w kodzie). Zero zmiany semantyki produkcyjnej. Unifikacja severity to już redesign kontraktu domeny — poza tą zmianą (osobna analiza modelowania domeny / DDD, nie refaktor struktury).

3) Historia gita mówi „guard, nie rebuild"

Commity wokół MNG-8294, maven.maven3Personality i MNG-5146 pokazują świadomą eskalację ostrzeżeń w M4 oraz flagę wstecznej osobowości. Research: to świadome ograniczenie, nie przypadek do „posprzątania".

4) Luka characterization jest konkretna i mierzalna

Mamy unit na FATAL + hint, mamy compat WARNING (MNG-5146), brakuje unit demotion przy personality i jasnej pary impl↔compat na tym samym układzie POM. To dziura, którą da się domknąć w 1–2 sesjach.

> Analogia
>
> Masz dwa regulatory prędkości: „nowy, surowy" i „stary, łagodny". Różnica jest celowa (bezpieczeństwo vs kompatybilność). K2 nie łączy ich w jeden. K2 wiesza tabliczkę i zakłada test, żeby ktoś w dobrej wierze nie „naprawił niespójności" i nie usunął różnicy.

8. Co konkretnie zrobimy (i czego nie)

Zakres planu — trzy fazy

Faza	Dostarcza	Uwaga
1. Contract note	Komentarz w kodzie + notatka kontraktu w folderze zmiany	Tylko dokumentacja intencji
2. Demotion unit	Test: z personality → WARNING; bez → FATAL	Na layoutcie WRONG_PARENT (jak MNG-5146)
3. Paired gate	Review pary impl↔compat; zero nowego kodu	Potwierdzenie twin, nie kopiowanie fixture

Świadomie poza zakresem

- Unifikacja FATAL↔WARNING między twinami (to byłby redesign kontraktu domeny)
- Extracty strukturalne K1 / K3
- Nowe SPI, migracja deprecated ProjectBuilder
- Jacoco jako cel; badanie ClassRealm / zewnętrznych ModelTransformer
- Korekty samego pliku repo-map.md

Kryterium sukcesu: kontrakt zapisany; testy chronią FATAL (default) i WARNING (personality) po stronie impl; compat WARNING zostaje; semantyka produktu bez zmian.

Review planu: werdykt SOUND (po synchronizacji briefu z fazą 3 jako verification gate).

9. Jak to się ma do całego M4 / „realizing change”

W ścieżce lekcji / pracy M4 widać świadomy ciąg:

- Mapa terenu (repo-map) — gdzie jest system i co boli.
- Deep focus (model-builder-flow-analysis) — co konkretnie jest długiem na hot path.
- Ranking i plan (refactor-opportunities) — co warto robić najpierw i jak, bez scope creep.
- Realizacja — małe, odwracalne kroki (guard-first), potem ewentualnie K1/K3 jako follow-up.

To jest właśnie „realizing change” w sensie inżynierskim: nie wielki bang refaktoru, tylko zmiana uziemiona w dowodach, z jawnym „czego nie robimy” i z najmniejszym możliwym blast radiusem. Dzięki temu AI agent / contributor nie „ulepsza” Mavena w sposób, który psuje most migracyjny M3↔M4.

Follow-upy (świadomie później): extract stage’ów K1, characterization+extract K3, pomiar pokrycia, research ClassRealm, adopcja personality w ekosystemie. Osobno — gdyby kiedyś „naprawiać” severity twinów — to już redesign kontraktu domeny, nie struktura.

10. Podsumowanie dla kogoś spoza zespołu

Problem biznesowo-techniczny: Maven 4 musi być ostrzejszy niż Maven 3 w niektórych regułach (np. niespójny parent), ale ekosystem nadal potrzebuje mostu i trybu „osobowości M3”. Różnica zachowań jest celowa, a brakuje tarczy testowej przed przypadkowym zatarciem tej różnicy.

Dlaczego ta zmiana teraz: mapa wskazała strefę budowania modelu jako #1 ryzyko; analiza długu wypisała okazje; ranking mówi „najpierw K2”; plan jest wąski, tani i chroni kontrakt przed dalszą pracą strukturalną.

Czego ta zmiana nie jest: nie jest przebudową architektury Mavena, nie jest unifikacją M3 i M4, nie jest „czyszczeniem 3450 LOC”. Jest ubezpieczeniem przed fałszywą poprawką.

> Zdanie na plakat
>
> Robimy refactor-opportunities, bo mapa i research powiedziały nam, gdzie Maven 4 jest najdelikatniejszy — a my zaczynamy od najmniejszego kroku, który chroni celową różnicę M3↔M4, zanim ktokolwiek ruszy duży hub.

Źródła (w repozytorium)

context/map/repo-map.md — mapa onboardingowa i strefy ryzyka

context/changes/model-builder-flow-analysis/ — prior długu ścieżki ModelBuildera

context/changes/refactor-opportunities/change.md — intencja i etapy zmiany

context/changes/refactor-opportunities/research.md — ranking K2 → K1 → K3

context/changes/refactor-opportunities/plan.md (+ plan-brief.md) — zakres K2 guards

context/changes/refactor-opportunities/reviews/plan-review.md — werdykt SOUND

Wygenerowano lokalnie na potrzeby notatek M4 · plik: .claude/notes/m4/m4-realizing-change-reasons.pdf